

Basic Shapes Project Report Template

Name: Emory Bruington

GitHub repository URL: <https://github.com/l4mont/csc-223-basic-shapes-project>

IDE used: Visual Studio 2026

Project Description

This program models circles, rectangles, and squares. BasicShape defines the common name, area, and calculation interface. Circle and Rectangle supply area formulas. Square inherits Rectangle and keeps all its dimensions equal. A single loop displays two circles, two rectangles, and one square. Unit tests check calculations, invalid input, automatic updates, and inheritance. The formal interface requires concrete shapes to provide area-calculation behavior.

Planning

BasicShape inherits from ABC, and `calc_area()` is abstract so incomplete shape classes cannot be created. Dimensions must be positive real numbers; circle coordinates may be negative or zero. Wrong types raise `TypeError`. Zero, negative, or nonfinite dimensions raise `ValueError`. Names must be nonblank strings. Each setter validates before changing state and then updates area. Circle and Rectangle inherit BasicShape; Square inherits Rectangle. Square keeps side, length, and width equal. Tests cover construction, validation, recalculation, failed assignments, and polymorphism.

Class-Hierarchy Plan

Class	Base Class	Protected Attributes	Public Properties	Area Behavior
BasicShape	ABC	<code>_name</code> , <code>_area</code>	<code>name</code> , <code>area</code>	Abstract <code>calc_area()</code>
Circle	BasicShape	<code>_x_center</code> , <code>_y_center</code> , <code>_radius</code>	<code>x_center</code> , <code>y_center</code> , <code>radius</code>	$\pi \times \text{radius squared}$
Rectangle	BasicShape	<code>_length</code> , <code>_width</code>	<code>length</code> , <code>width</code>	$\text{length} \times \text{width}$
Square	Rectangle	<code>_side</code> ; inherited dimensions	<code>side</code> , <code>length</code> , <code>width</code>	Inherited <code>Rectangle.calc_area()</code>

Design

The formal ABC interface explicitly connects the classes and requires area-calculation behavior. A duck-typed design would only assume that unrelated objects provide the needed properties. Square inherits Rectangle because its area uses the same formula. Its length and width setters both delegate to side, so changing any dimension resizes the entire square. BasicShape provides the shared name and area properties. Circle represents a center and radius; Rectangle stores length and width. Property setters protect valid object state.

Implementation

Each class has its own file and imports its base class. Circle and Rectangle call `super().__init__(name)`. Square calls Rectangle's constructor with two equal dimensions. BasicShape starts area at 0.0 without calling a subclass formula. Rectangle delays its first calculation until both dimensions exist; Circle calculates after storing radius. Properties validate values, preserve state after rejected assignments, and recalculate automatically. The area property is read-only. Circle.`calc_area()` uses `math.pi`; Square reuses Rectangle.`calc_area()`. Setters also reject values whose calculated area becomes infinity or zero. Modules, classes, properties, and methods have docstrings.

Unit Testing

This project requires formal unit testing with Python's unittest module. Do not substitute manual inspection testing for the required unit tests.

Testing Environment

Visual Studio 2026 used its Python 3.14 interpreter. BasicShapes.pyproj selects main.py and unittest discovery in tests. Test Explorer discovered and ran 43 tests in five files. Earlier terminal testing used Python 3.12.14. The terminal command is `python -m unittest discover -s tests -v`.

Unit Testing Strategy

Test classes inherit `unittest.TestCase`; methods start with `test_`. Tests arrange an object, perform an operation, and check the result. `assertEqual` checks exact values, `assertAlmostEqual` checks decimal areas, and `assertRaises` checks exceptions. Tests cover the abstract class, names, read-only area, invalid dimensions, recalculation, failed assignments, square equality, and mixed-shape processing. Tests were written after the initial implementation and rerun after the area-range correction. Direct BasicShape construction and an incomplete subclass must raise `TypeError`. Dimension changes must update area; square updates must keep all dimensions equal. A mixed list checks the common interface.

Required Test Results

Test or Feature	Input or Initial State	Expected Result	Actual Result	Pass/Fail	Correction or Comment
Instantiate BasicShape	Direct construction	TypeError	TypeError	Pass	Abstract class enforced
Valid Circle	Positive radius	Correct area	50.26548	Pass	Initial area
Zero radius	0	ValueError	ValueError	Pass	Construction rejected
Negative radius	Negative value	ValueError	ValueError	Pass	Construction rejected
Nonnumeric radius	String or other invalid type	TypeError	TypeError	Pass	Each value tested
Circle radius change	New positive radius	Area recalculated	201.06193	Pass	Automatic update
Circle coordinate change	New x or y value	Area unchanged	16 × pi retained	Pass	Center updated

Test or Feature	Input or Initial State	Expected Result	Actual Result	Pass/Fail	Correction or Comment
Valid Rectangle	Positive dimensions	Correct area	200	Pass	Initial area
Invalid Rectangle length	Negative value	ValueError	ValueError	Pass	Width tested too
Rectangle dimension change	New length or width	Area recalculated	600; 400	Pass	Independent cases
Valid Square	Positive side	Side = length = width	(10, 10, 10)	Pass	Area 100
Square side change	New side	All dimensions synchronized	(20, 20, 20)	Pass	Area 400
Square length/width assignment	New valid value	Square invariant preserved	All 7; all 5	Pass	Areas 49 and 25
Failed dimension assignment	Invalid new value	Previous valid state preserved	Previous state retained	Pass	All shape classes
Mixed shape list	Circle, Rectangle, Square objects	Processed through common interface	All five verified	Pass	No type chain
Full unittest suite	All test files	All required tests pass	43 passed; 0 failed	Pass	No errors or skips

Problems and Corrections

The initial implementation checked dimensions but not the resulting area. Assigning radius 1e308 produced infinity; 1e-300 produced zero. The numeric-range tests expected ValueError and an unchanged object, so they failed. The correction checks the prospective area before storing dimensions. The test_unrepresentable_area methods for Circle, Rectangle, and Square then passed. Tests were added after the initial implementation. The failing cases were corrected and rerun, followed by the full suite. Evidence is saved in evidence/initial-tests.txt and evidence/final-tests.txt. Correction commit: 3bbd9b0 (Fix area overflow and underflow validation).

Final Test Status

43 tests passed; 0 failed. All required abstraction, validation, recalculation, Square-invariant, and polymorphism checks passed. Visual Studio also ran main.py and displayed the expected shape areas and dimension updates. The Debug output reported that MainThread exited with code 0.

Reflection

1. How did the abc module make the interface explicit?

ABC and @abstractmethod declare that every concrete shape must provide or inherit calc_area().

2. What happens if a direct subclass of BasicShape does not implement `calc_area()`?

It remains abstract. Trying to create an instance raises `TypeError`, as the tests confirm.

3. How would an informal duck-typed design differ from the formal interface used in this project?

Duck typing would accept any object with the expected properties or methods. This project requires an explicit `BasicShape` inheritance relationship.

4. What data and behavior did `Circle` and `Rectangle` inherit from `BasicShape`?

They inherit `name` and `area` properties, `name` validation, the numeric-validation helper, and the `calc_area` contract. The base constructor creates `_name` and `_area`.

5. Why does `Square` inherit from `Rectangle` rather than directly from `BasicShape`?

It can reuse the length-and-width representation and product formula while adding the equal-dimension rule.

6. Why can `Square` reuse `Rectangle.calc_area()`?

When length and width both equal side, length times width is side squared.

7. How did you preserve the `Square` invariant when side, length, or width changed?

The side setter validates first, then updates side, length, and width together. The inherited dimension setters delegate to side. Invalid updates leave every value unchanged.

8. How did runtime polymorphism allow different shape objects to be processed through one common interface?

One loop reads each shape's name and area. Calls to `calc_area()` resolve to `Circle`'s or `Rectangle`'s method based on the actual object; `Square` inherits `Rectangle`'s method. No type-checking chain chooses a formula.

9. How did properties maintain valid dimensions and correct areas?

Setters reject invalid input before modifying the object and recalculate after valid changes. Clients cannot directly assign area.

10. How did `unittest` change the way you verified that the class hierarchy worked?

Repeatable assertions checked exact expectations, including invalid values and state preservation. These checks found a boundary problem that ordinary demonstration values missed.

11. Describe one failed unit test that helped you improve the program.

`TestCircle.test_unrepresentable_area` showed that radius `1e308` was accepted even though its area became infinity. Checking the new area before assignment fixed the defect; matching tests cover rectangles and squares.

12. How did docstrings help explain the class hierarchy, interface, and invariants?

They explain each class's role, valid inputs, exceptions, and automatic recalculation. Square's documentation makes its synchronized resizing behavior clear.

13. What did you learn about inheritance, polymorphism, formal interfaces, and unit testing in Python?

Inheritance saves repeated code, but a subclass must preserve its own rules. ABC states the required behavior, properties protect state, and tests check that the pieces work together.

Important Notes

- Write the report in your own words.
- Keep the report IDE neutral.
- Focus on design, inheritance, formal interfaces, polymorphism, validation, and unit testing.
- Do not paste large sections of source code into the report.
- Use the results from your unittest suite rather than manual inspection as the primary testing evidence.
- Submit the completed report in the format specified by the instructor.